PRODUCT REQUIREMENTS DOCUMENT

BidOn’s Telemetry Project

Status	Complete
Owner	Jonathan Kuperberg
Last Updated	October 14, 2026
Target Release	[Q4 2026]
Platforms	Android
Availability	All publishers via BidOn SDK integration
Organization	BidOn Foundation — 501(c)(3) non-profit

Summary

Bidon is instrumenting the full ad lifecycle across all three of its integration modes (publisher-direct, AppLovin MAX as a certified bidding partner, and AppLovin MAX as a custom network adapter) with a single event schema that treats host and auction ownership as dimensions rather than forks.

The immediate driver is that Bidon is about to start rendering third-party DSP creatives itself, in someone else's mediation stack, on hardware it doesn't control. That is a category of risk Bidon has never carried, and it is currently unobservable. The strategic driver is that delivery evidence is a precondition for both mediator certification and DSP acquisition (the two things the roadmap depends on).

Part I below covers context, problem, rationale, goals and scope. **Part II** is the detailed event catalog and instrumentation spec.

Context

1. **Bidon runs in three modes, and they are not variations**

   Bidon ships one SDK that operates in three fundamentally different postures:

   - **Direct Mode** (Current Product): Publishers integrate the Bidon SDK directly. Bidon is not a mediator and does not run a waterfall of third party network SDKs. The SDK requests an ad from the bidon server, the server runs an auction among DSP partners over oRTB, and the SDK renders the winning creative. Bidon owns the latency budget and the publisher relationship, and owns fill in the sense that exchange either returns a renderable bid or does not.
   - **Max Bidding Mode:** Bidon is certified as a single bidding partner inside AppLovin MAX, giving multiple DSP partners access to MAX auctions without each negotiating its own certification. Here Bidon is one bidder among many. MAX owns the auction, fills, fires the notifications, and sits between the Bidon SDK and the Bidon server.
   - **Max Custom Mode:** Bidon configured as a custom network in a publisher's MAX waterfall at a manually set CPM. MAX calls the adapter to load an ad but hands over no bid response, so Bidon runs its own auction internally and renders the result

   These modes differ in who owns the auction, who fires nurl/burl/lurl, who is responsible for fill, and critically, how much of the chain Bidon can see at all.

2. **Three things changed at once**

   a. **Bidon started rendering creatives itself.** The rendering architecture moves Bidon from orchestrating other SDKs' renderers to being the renderer, that means when a DSP's creative fails to display, it is Bidon's bug in Bidon's renderer across arbitrary creative payloads from partners Bidon does not control.

      The same shift transfers **measurement** ownership, not just display. MAX's Open Measurement integration covers creatives rendered by the AppLovin SDK;

creatives rendered by the Bidon uSDK are outside it. That is the premise of the parallel **OM SDK integration spike**, and it means Bidon now owns two measurement systems on one rendering surface (verification (OM) and observability -> this project)

b. **Bidon introduced a non-standard `adm` envelope.** DSPs must wrap creatives in Bidon's JSON envelope carrying `dsp_id`, `crtype` and `render_config`. This is a breaking change from standard oRTB and every DSP partner has to implement it.
c. **Bidon is entering a certification process.** Becoming a certified bidding partner in MAX means a counterparty will be measuring Bidon's delivery and comparing it against their own numbers.

Problem Statement

Bidon cannot currently answer basic questions about its own delivery in any of the three modes.

- Why did we not participate in an auction?
- Why did a won bid not become an impression?
- Which DSP's creatives fail to render, and how often?
- Where is the latency budget going?
- What did we actually earn versus what the mediator says?
- Is this publisher's complaint a Bidon defect or an integration mistake?

This is a structural gap, not a tooling gap. These questions are unanswerable without instrumentation, no amount of log-diving, better dashboards on existing data, or querying MAX's reporting recovers them. In particular, the silent-timeout failures produce no artifact at all: the absence of a bid request is the only evidence, and absence is not attributable.

The current cost is that every integration issue becomes a manual investigation with a partner on the other end, and some classes of revenue loss (auctions never entered, wins never rendered) are not merely hard to quantify but invisible.

Strategic rationale

1. **Delivery evidence is a certification input, not a post-launch nicety**

   Mediator certification processes involve the mediator measuring the partner's delivery and reconciling it against the partner's own numbers. A bidding partner that cannot produce impression-level data to compare is not in a position to resolve a discrepancy dispute (it can only accept the mediator's figures)

   Telemetry is therefore an input to certification, not something that follows it.

2. **Bidon has taken on rendering risk it has no history with**

   The renderer is new code, executing untrusted third-party creative payloads, inside a host mediation SDK, across the full Android device and OS distribution. Every prior generation of this risk was absorbed by network partners' own SDKs.

   Shipping that renderer without instrumentation means the first signal of a systematic rendering defect is a partner or publisher complaint, which arrives late, without diagnostics, and to the counterparty whose trust the certification depends on.

3. **Direct mode is the proving ground for the MAX launch**

   Because the renderer, the `adm` envelope, the DSP demand and the creative pipeline are **identical in both modes**, direct-mode traffic can validate the renderer at volume before AppLovin is asked to trust it. If render rate holds at 99%+ across real direct-mode traffic, that is evidence Bidon can bring to the certification conversation rather than a promise

4. **DSP acquisition depends on being able to show delivery**

   Per-DSP render rate, loss reason codes, latency and discrepancy figures are **commercial assets**, not just ops tooling.

   Viewability is part of the same evidence package. Brand demand routinely requires third-party verification, so a bidder that cannot offer OMID measurement is closed off from some categories of spend regardless of how well it renders, which is why the OM SDK spike and this project are commercially coupled even though they are technically separable.

5. **Open source raises the standard**

   Bidon's telemetry implementation is publicly readable. What is collected, what is not, and how consent gates it are visible to every publisher evaluating the SDK and every competitor. A clean, minimal, well-gated telemetry design is part of the neutrality and trust proposition; a sloppy or over-collecting one is a reputational liability that cannot be quietly fixed.

6. **Cost of delay is asymmetric, and SDK release lag is the reason**

   Server-side instrumentation can be added retroactively at any time. **Client-side instrumentation cannot.** An event missing from the SDK today is missing until a new SDK version ships *and* publishers upgrade (a propagation curve measured in quarters, not weeks, and never reaching 100%).

7. **Custom adapter mode is a de-risking and negotiating instrument**

`max_custom` should not be evaluated on its own commercial merits, where it is clearly inferior (static CPM, no bidding, no auto-CPM). Its value is positional, and it strengthens three arguments already made above.

- **It is a better proving ground than direct mode:** Direct-mode traffic validates the renderer, but in Bidon's own host environment. `max_custom` validates the renderer and the MAX adapter code path *and* the MAX SDK host environment, on real MAX inventory
- **It converts the certification conversation from promise to evidence:** Walking into a certification discussion able to say "here is our render rate, latency distribution and impression count across N million impressions of your inventory, in your SDK" is a different conversation from presenting a design
- **It is the fallback if certification stalls:** Certification depends on a counterparty's timeline and willingness. `max_custom`. Applovin still hast to make Bidon available as a custom network option. Telemetry built in 2 axis model works identically in it, so there is no rework if the fallback becomes the plan.
- **The cost to telemetry scope is small** because `max_custom` reuses the `direct` event set almost entirely.

Goals

#	Description
1	Make every stage of the ad lifecycle observable in all three integration modes, from SDK init to billing.
2	Attribute every failure to a domain (Bidon SDK, Bidon server, mediator, network adapter, DSP, publisher integration, device) with a stable error code and the foreign system's original code preserved
3	Produce impression and revenue data that reconciles with mediator reporting within an agreed tolerance
4	Provide per-DSP delivery and loss data suitable for partner-facing reporting
5	Establish a schema that extends to additional mediators without redesign
6	Surface latency per leg so budget regressions are attributable, not just visible

Users and use cases

User	Use case	Event Examples
SDK engineers	Diagnose render and load failures	`adm_parse_result`, `ad_load_failed`, `webview_error`, `ad_impression`
Server engineers	Tune DSP timeout budgets; monitor auction health	`dsp_response_received`, `auction_completed`
Partner/solutions engineers	Onboard DSPs onto the `adm` envelope; triage partner escalations	`adm_parse_result.unknown_fields`, `dsp_response_rejected`
Publisher support	Separate Bidon defects from integration mistakes	`error_domain=publisher`
BD / Partnerships	Per-DSP delivery evidence in acquisition conversations	render rate by `dsp_id`, discrepancy
AppLovin	Reconciliation during and after certification	`ad_impression`, billing events
DSP partners	Bid optimisation from loss data; delivery confirmation	`loss_notice_*`, per-DSP render rate
Publishers (direct mode)	Fill rate, time to fill, revenue	`ad_filled`, `waterfall_exhausted`

Success Metrics

These measure whether the telemetry project succeeded, distinct from the ad-performance metrics that the telemetry produces. Targets are proposals to calibrate against a baseline.

- **Funnel completeness:** Ad requests with an unbroken event chain from request to terminal outcome

    - Proposed Target > 98%
- **Attribution rate:** Failures carrying a valid `error_domain` + `error_code` (not "unknown")
    - Proposed Target > 95%
- **Time to diagnose:** Median time from issue report to root-cause domain classification
    - Proposed Target -> Baseline, then ≥ 50% reduction
- **Self-service resolution:** Escalations resolved from telemetry without reproducing the issue
    - Proposed Target > 70%
- **Reconciliation delta:** Bidon-reported vs mediator-reported impressions
    - Proposed Target -> Within agreed tolerance, and explainable when outside it
- **Telemetry loss rate:** Events dropped client-side (`telemetry_dropped` ÷ emitted)
    - Proposed Target < 1%
- **Ad-path latency impact:** p95 ad request duration, instrumented vs control
    - Proposed Target -> No Measurable regression.

Scope and Milestones

Milestone	Mode	Scope	Unblocks
M0 - Foundation	N/A	`host` + `auction_owner` model, event envelope, correlation chain, error model, client transport	Everything. Includes the `adm` envelope change, which must land before more DSPs implement
M1 - Direct Integration	Direct	~80% of the total surface: lifecycle, token collection, auction, waterfall and fill, load and render, outbound notifications	Fill rate and time to fill; the baseline render rate every later milestone is compared against
M2 - Custom Adapter	max_custom	Thin delta: borrowed-timeout events, `configured_cpm`, two metrics	MAX inventory without certification; the delivery evidence that makes M3's certification conversation concrete
M3 - Certified Bidding	max_bidding	Signal collection, `adm` correlation, MAX bid-request handling, inbound notifications	Certification reconciliation and per-DSP loss data

The event set is cumulative, not sequential. M1 is built once and reused unchanged in M2; M3 adds the MAX handoff and inverts notification direction. Nothing is rebuilt at any step

Risk and Mitigation

Risk	Impact	Mitigation
Telemetry degrades ad-path latency	N/A	`host` + `auction_owner` fire and forget, off the main thread, enqueue never wait, p95 regression gate in success metrics
Direct-mode event volume (20–40× per impression) blows up cost	Budget overrun; ingest instability	Sampling with rate recorded on-event; mode-specific rates; plan capacity against direct-mode volume
`ad_impression` on the revenue path couples billing to telemetry reliability	Under-billing if events are lost	Impression carried on the ad-serving channel with delivery guarantee, not the best-effort batch queue
Privacy exposure, amplified by public repo	Regulatory and reputational	Consent gating, no PII, scrubbed error messages, legal review before launch, minimal collection as a design constraint
Schema churn breaks downstream queries	Analysis rework	`schema_version` on every event; additive-only changes within a major version
Phase 0 slips past DSP onboarding	Breaking change to partners who already integrated	Sequence Phase 0 ahead of DSP envelope rollout
Scope creep into a reporting product	Delivery delay	Explicit non-goal
Bidon's billing impression and	DSPs billed on one number, verified by advertisers on	Single impression definition owned by the OM spike, adopted by telemetry

OM's measured impression diverge	another (the kind of gap that gets spend paused)	
OM ships in a different SDK release than telemetry	Version-gated cohorts make viewability un-analysable	Ship the shared render-surface instrumentation together
OM redistribution blocked by IAB licensing, so OM is absent from some builds	Viewability rates uninterpretable ("not viewable" indistinguishable from "not measured")	`om_enabled` on every impression from day one
`max_custom` internal waterfall overruns MAX's adapter timeout	Wasted MAX latency; publishers deprioritise or remove the custom network	Instrument `adapter_load_budget_exceeded`
`max_custom` and `max_bidding` data blended in certification evidence	Misleading figures presented to AppLovin	Two-axis model in Phase 0
`configured_cpm` unreadable from MAX server parameters	Custom-adapter yield cannot be tuned	Confirm adapter parameter access early
Client event gaps discovered post-release	Unfixable for quarters	Over-emit in Phase 1 relative to current analysis needs
A broad envelope multiplies payload across every event	Cost and bandwidth compounding with the volume risk above	Tier the envelope: A common core on every event, field group only on the event classes where they are meaningful
Bid responses discarded by	Bid rate looks Healthy while win rate is inexplicable low	bid_timeout_suspected, instrumented against MAX’s documented server-side

MAX as late go unobserved		bid timeout.

Event Catalog and Instrumentation Spec

Based on previous milestone definition, following we can identify what need to be considered in each of them

M0 - Foundation

- **Common event envelope**

  Present on every event, all milestones. Nullable where the stage hasn't produced them.

Field	Type	Notes
event_name	string	
event_id	uuid	Dedupe key
event_ts	int64 ms	Wall clock at emission
schema_version	int	
host	enum	publisher / max_adapter
auction_owner	enum	bidon / max
integration_mode	enum	Derived label: direct / max_custom / max_bidding.

mediator	enum	max / bidon (extensible to level_play, admob)
mediator_sdk_version	string	Null when host = publisher
session_id	uuid	Groups all activity in an app session, all modes
auction_id / bid_id / dsp_id	uuid/string	Correlation chain -> See note 1
token_id	uuid	M3 only
demand_source	string	Adapter or DSP that produced the ad
demand_type	enum	bidding / direct_deal. There is not network demand. Bidon does not mediate third party network SDKs
sdk_version	string	
app_bundle, app_version, publisher_id	string	
os, os_version, device_model, device_manufacturer	string	
connection_type	enum	wifi / cellular_4g / cellular_5g / offline / unknown
country	string	Server-derived from IP; never sent from client
ad_format	enum	banner / mrec / interstitial / rewarded / native
ad_unit_id, placement	string	MAX ad unit when host = max_adapter; Bidon placement when host = publisher
gdpr_applies, has_tcf_string, us_privacy, coppa, lmt	bool/string	Check Note 2
consent_source	enum	bidon_cmp / max_sdk / publisher_api (consent

		provenance differs by host, and a mismatch between what MAX passes and what Bidon holds is a compliance defect that is undiagnosable without this field)
sampling_rate	init	
sdk_uptime_ms	int64	Orders events when device clocks are wrong
session_start_ts	int64	Session duration without a self join
ab_test_group	string	
wrapper_version	string	
os_api_level	int	
device_type	enum	oRTB device type codes
screen(width, height and density)	int/float	to support render bugs correlate with density and unusual aspect ratios
gpp_string, gpp_consent	String / bool	GPP is current US framework and is replacing the US privacy string, which the envelope already carried
mspa_enforced	bool	Travels with GPP
dnt	Bool	a separate signal from limit_ad_tracking
Advertising ID	String	The device advertising identifier (GAID) Populate only when the consent is present, otherwise is zeros (never null or omitted)
Advertising_id_state	enum	The value is identical across all three zeroes.

- **`auction_id` / `bid_id` in the `adm` envelope:**

These fields are only *needed* when MAX sits in the middle. But adding them is a breaking change to the DSP-facing schema, and DSPs are onboarding onto that schema now. Every partner who implements before the field lands is a partner who has to change. The telemetry justification is Milestone 3; the scheduling justification is not wanting to break partners twice.

- **`om_enabled` and the render-surface fields:**
  If the OM SDK is absent from some builds, a null viewability rate is ambiguous: *not viewable* and *not measured* become indistinguishable, and every viewability number is uninterpretable retroactively

- **Error model:**
  - **`error_domain`** (who owns the failure): bidon_sdk, bidon_server, max, network_adapter, publisher, dsp, network, creative, device
  - **`network_adapter`** (failures inside a third-party network SDK).
  - **`publisher`** (integration mistakes): `show()` before `load()`, no activity context, wrong thread, missing placement config, ad shown twice, and in M2, MAX custom-network misconfiguration.
  - **`error_code`** (stable integers, never reused, banded by stage):

Band	Stage	Milestone Needed
1000–1999	SDK init / config	M1
2000–2999	Token collection (both directions)	M1
3000–3999	Server: request handling	M1
4000–4999	Server: DSP communication	M1
5000–5999	`adm` envelope parsing	M1
6000–6999	Renderer / creative load	M1
7000–7999	Display / impression	M1
8000–8999	Network / transport	M1
9000–9999	External (wrapped MAX / DSP / network-SDK errors)	M1
10000–10999	Auction and waterfall	M1

11000–11999	Publisher / host integration errors	M1 (Extended in M2)

- **`external_error_code`** + **`external_error_source`** (`max_sdk`, `max_server`, `dsp`, `webview`, `http`, `network_adapter:<name>`) always preserved raw, never normalised away.
- **Rules:** errors are never sampled; every error still emits the terminal event for its funnel stage so drop-off arithmetic stays consistent.
- The silent failure patterns: A recurring class of failures in this system. Bidon is dropped without an error being raised on Bidon's side. The only evidence is an absence, and the absence is not attributable. Each instance requires a budget-aware timer of our own, because the external system returns nothing

Instance	Milestone	Instrumentation
Borrowed MAX adapter budget exceed	M2	Adapter_load_budget_exceeded
CollectSignal() outside MAX budget	M3	Signal_timeout_suspected
Bid response delivered after MAX's server side timeout	M3	bid_timeout_suspected

Note 1 (Correlation Chain): In Milestone 1 and 2, The SDK requests the auction from the Bidon server itself, so the server mints `auction_id` and returns it in the auction response. In Milestone 3, Client and server never talk during an auction, so IDs must ride inside payloads MAX already forwards

Note 2 (Consent fields): These are the passthrough values, not Bidon decisions. Bidon's role is strict passthrough of the publisher or mediator signal. The fields are recorded so a downstream filtering decision can be explained, and consent_source records which system supplied them. Sub-forwarding consent is visible as CPM softness: Over-forwarding is invisible until external discovery, so provenance maters more than the value itself

M1 - Direct Integration

`host = publisher`, `auction_owner = bidon`. The existing product.

- **SDK lifecycle:**

Event	Fired When	Key Fields
sdk_init_started	BidonSdk.initialize() entered	init_trigger
sdk_init_completed	Init finished	duration_ms, adapters_registered[], config_source
sdk_init_failed	Threw or timed out	error_code, failed_phase, duration_ms
adapter_init_result *	Per adapter	adapter_name, adapter_version, success, duration_ms, error_code, external_error_code
config_fetch_result	Remote config call	success, duration_ms, http_status, from_cache

`*adapter_init_result` is load-bearing wherever **auction_owner = bidon:** There are no third-party network SDK to initialize, because bidon does not mediate. This event covers initialization of Bidon's own components and of any DSP-side integration that requires client initialization.

- **Auction:**
    - Client Side

Event	Fired When	Key Fields
ad_request_started	Publisher calls `load()`	placement, ad_format, pricefloor, is_autorefresh, cached_ad_available, trigger
auction_requested	SDK posts auction request	auction_id, pricefloor, token_count, adapters_included[]
auction_response_received	Server responds	duration_ms, demand_count,

		top_ecpm, response_size_bytes
auction_failed	No response / error	duration_ms, error_code, http_status
auction_no_demand	Valid response, zero eligible demand	pricefloor, reason

- Server Side

Event	Key Fields
auction_request_received	auction_id, placement, pricefloor, ad_format, country, bundle
dsp_request_sent	dsp_id, timeout_budget_ms, endpoint
dsp_response_received	dsp_id, latency_ms, outcome (bid/nobid/timeout/http_error/malformed), price, nbr, http_status, creative_id, crtype, supports_omid
dsp_response_rejected	dsp_id, reason (below_floor/schema_invalid/missing_adm_fields /blocked_category/expired)
auction_completed	winner_demand_source, clearing_price, participant_count, total_latency_ms

- **Load and Render:**
  - Load

Event	Key Fields
ad_load_requested	caller (publisher_api / max_adapter), has_bid_response
adm_parse_result	success, auction_id, bid_id, dsp_id, crtype, envelope_version, unknown_fields[], fallback_applied, duration_ms

renderer_selected	renderer (html/vast/mraid), crtype, selection_reason
ad_loaded	total_duration_ms, renderer, creative_size_bytes
ad_load_failed	error_code, error_domain, duration_ms, failed_stage*
ad_expired	ttl_ms, age_at_expiry_ms

`*failed_stage` enum (parse / renderer_select / asset_fetch / webview_init / webview_load / timeout), because "load failed" without a stage is not actionable

- Display

Event	Key Fields
ad_show_requested	time_since_loaded_ms
ad_impression	render_duration_ms, om_enabled
ad_show_failed	error_code, failed_stage
ad_viewable	time_to_viewable_ms, om_enabled, om_partner_name
ad_clicked	click_target, time_since_impression_ms
ad_closed	time_visible_ms, close_method (user/auto/app_backgrounded)
ad_reward_granted	Rewarded only
webview_error	error_code, description, failing_url_host, is_main_frame
render_crashed	renderer, crtype

VAST quartiles (`video_start`, `video_q1`, `video_midpoint`, `video_q3`, `video_complete`, `video_skipped`, `video_error`) arrive with the video renderer phase, in all modes.

- **Interface with the OM SDK integration:**

  The OM SDK spike and this project are separate deliverables (different skill sets, and OM carries a licensing blocker telemetry does not). But they instrument the same rendering surface, and treating them as unrelated produces two incompatible definitions of the same events inside one SDK.

  Following are the fields that it will requires to collect regardless the output of the OM spike project

Field	On which events	Why
om_enabled	every impression	Without this you cannot distinguish *not viewable* from *not measured*, retroactively and permanently
om_partner_name, om_sdk_version	every impression	OM ships as a namespaced build per partner; which build shipped is a real variable
om_session_error	on failure	OM session failures are silent to the user and invisible in ad metrics
om_measurement_mode	every impression	Native measurement vs verification-script injection give non-comparable results
supports_omid	dsp_response_received	Per-DSP OMID readiness

- **Notification (outbound):**
  - Creative and error fields on client events

Field	On which events	why
Creative_id	client_load / render / Display	currently server-side only on dsp_response_received. Without it on the client_events a render failure cannot be joined to the creative that

		caused it. which leaves a stated project goal unanswerable
partner_creative_id	same	The DSPs own ID, which differs from ours and is what they quote in a dispute
Requested_width, requested_height	ad_request / Load	what the publisher asked for, taken from the slot the sdk sized
delivered_widht, delivered_height	render / impression	What the creative actually is, measured by the render after load
requested_media_type, response_media_type	load	response_media_type is the existing crtype. The pairing is what makes mismatch detectable
impression_id	impression	The DSPs own impression id fro the creative payload, which is what a partner quotes in a discrepancy dispute
error_cause	any failure event	The underlying exception.

Wherever `auction_owner = bidon`, Bidon must notify DSPs. This is a server responsibility, not just telemetry, and its reliability is Bidon's problem: a DSP that doesn't receive loss notices can't optimise bids, and one that doesn't receive billing notices will report a spend discrepancy.

Event	Key Fields
win_notice_sent	dsp_id, clearing_price, success, latency_ms, http_status, retry_count
billing_notice_sent	dsp_id, billable_price, success, http_status, retry_count
loss_notice_sent	dsp_id, loss_reason_code, winning_price, success,

	http_status
notice_delivery_failed	dsp_id, notice_type, error_code, attempts

- **Funnel and Metrics:**

Ad_request_Started -> Token_collection_completed -> Auction_requested -> Auction_response_received -> Demand_load_attempt (1..n) -> Ad_filled -> Ad_show_requested -> Ad_impression -> Billing_notice_sent

Metric	Definition
Adapter token success rate	per adapter, outcome=ok ÷ requested
Adapter init success rate	per adapter
Bid rate	auction responses with ≥1 demand ÷ requests
DSP timeout rate	per dsp_id, outcome=timeout ÷ requests (directly tunes the fan-out budget)
Fill rate	ad_filled ÷ ad_request_started (the headline publisher-facing number)
Time to fill	p50/p95 ad_request_started → ad_filled
Per-source load rate	attempts succeeded ÷ attempted, per demand_source (catches high-bid/low-deliver demand)
Render rate	ad_impression ÷ ad_filled (the baseline every later milestone is compared against)
Load failure rate by code	grouped by error_code
Publisher-error rate	error_domain=publisher ÷ ad requests (an integration-quality metric, not a defect metric)
Notice delivery rate	*_notice_sent success ÷ attempted
Size mismatch Rate	delivered_width/height differs from requested_width/height divided by impressions
Per-DSP delivery rate	render attempts succeeded divided by attempted.

	Catches a DSP that bids high and delivers badly

M2 - Custom Adapter

`host = max_adapter`, `auction_owner = bidon`. Architecturally M1 wearing a MAX adapter shell. Delta over M1 only

- **What change from M0/M1**
    - **Unchanges from M1:** SDK lifecycle, token collection, auction (client + server), waterfall, load and render, outbound notifications, the entire error model
    - **Changes from M1:**host flips to max_adapter; ad_request_started.trigger = max_custom_adapter; ad_unit_id carries the MAX ad unit alongside the Bidon placement; consent_source = max_sdk;
    - **New:** Two events, two fields, two metrics (See below)
- **New Events**

Event	Fired When	Key Fields
adapter_load_budget_exceeded	Own timer passes the configured MAX adapter timeout, whether or not work continues	elapsed_ms, budget_ms, stage_at_overrun *, attempts_made
no_fill_returned_to_max	Bidon returns no-fill so MAX falls through	reason, attempts_made, total_duration_ms

* `stage_at_overrun` says whether the budget went on token collection, the auction round trip, or too deep a waterfall, and each has a different fix.
`no_fill_returned_to_max` is a terminal outcome with no M1 or M3 equivalent. In M1 a no-fill goes to the publisher, and in M3 a win always produces a load attempt. It must exist or the fill-rate denominator is wrong.

- **New Fields**

Field	On	Notes

configured_cpm	envelope	The CPM set in the MAX dashboard, if the adapter can read it from server parameters.
stage_at_overrun	adapter_load_budget_exceeded	Enum matching the M1 latency legs

- **New Metrics**

Metric	Definition
Adapter budget overrun rate	adapter_load_budget_exceeded ÷ ad_request_started (predicts being deprioritised or removed by publishers)
Effective vs configured CPM *	mean clearing_price ÷ configured_cpm

* Effective vs configured CPM reads in both directions:
  - Consistently clearing **above** the configured CPM → Bidon is placed too low in MAX's waterfall and the publisher is under-monetising; raise the configured price
  - Consistently clearing **below** it, or high no-fill → Bidon is placed too high, burning MAX waterfall latency on requests it can't serve; lower it

M3 - Certified Bidding Partner

`host = max_adapter`, `auction_owner = max`. Delta over M1 only. MAX now owns the auction, so a substantial block of M1 stops firing and a new block appears.

What stops firing (Migrate from M2 into M3 stage)

Fate	Events	M3 Destination
Invert	adapter_token_requested, adapter_token_result, token_collection_completed	signal_*.MAX collects
Relocate	auction_requests_received, auction_completed	bid_request_received, bid_response_sent (Same component, different

		endpoint)
Dissolve	ad_request_started, auction_requested, auction_response_received , auction_no_demand_auction _failed	N/A
Invert	win_notice_sent, billing_notice_sent, loss_notice_sent, notice_delivery_failed	*_notice_received, notice_forwarded_to_dsp
Dissolve	demand_load_attempt_started, demand_load_attempt_result _ad_filled, ad_request_completed	
M2 only	adapter_load_budget_exceeded, no_fill_returned_to_max	

- **Signal Collection (outbound):**

  If `collectSignal()` misses MAX's timeout, Bidon is dropped from the auction and no error is raised anywhere.

Event	Fired When	Key Fields
signal_requested	MAX calls `collectSignal()`	
signal_provided	Token returned	duration_ms, token_id, token_size_bytes, gaid_available, token_source (fresh/cached)
signal_failed	Exception / internal timeout	duration_ms, error_code
signal_timeout_suspected	Own timer exceeds MAX's budget, even if a token is eventually produced	duration_ms, budget_ms

- **Server (Bid Request Handling):**

Event	Key Fields
bid_request_received	auction_id, token_id, ad_format, bundle, country, ortb_version, request_size_bytes
token_decode_result	success, error_code, token_age_ms *, token_schema_version
bid_response_sent	auction_id, seat_count, top_price, total_latency_ms, envelope_version
no_bid_sent	auction_id, reason, total_latency_ms
bid_timeout_suspected	Own timer : Total_latency_ms_exceeds

* `token_age_ms` measures how long MAX holds a token before using it. Upward drift means cached device signals are going stale and targeting quality is degrading.

- **Notification (inbound):**

  MAX fires these server-to-server; the SDK is never involved.

Event	Source	Key Fields
win_notice_received	nurl	auction_id, bid_id, dsp_id, clearing_price, latency_from_response_ms
billing_notice_received	burl	auction_id, bid_id, billiable_price
loss_notice_received	lurl	auction_id, bid_id, loss_reason_code, winning_price
notice_forwarded_to_dsp	relay	dsp_id, notice_type_success_latency_ms, http_status
unbilled_win*	batch job	auction_id, clearing_price,

		age_min, nurl_seen

* is the revenue-leak detector. A win that never bills means the creative failed to render on device, so its rate should track client side render failures closely.

- **ad_impression changes roles in this milestone:** In previous stages, ad_impression is the outcome signal that trigger billing_notice_sent. In M3 it comes off the revenue path entirely. MAXs burl is the authoritative and billling_notice_received is the revenue signal. ad_impression must still never sampled in M3, but because it is the reconciliation number against MAX, not because it triggers billing.

Transport and non-functional

Client

- Telemetry is fire-and-forget, off the main thread. It must be structurally impossible for the telemetry path to delay or fail an ad request: ad-path code enqueues, never awaits.
- **Exception: `ad_impression` in M1 and M2** only, where it is the outcome signal the server uses to decide whether to fire burl, and needs a stronger delivery guarantee than the batch queue. In M3 it comes off the revenue path, because MAXs burl is the authoritative.
- Disk-backed queue surviving process death; flush on batch size, timer, and app-backgrounded.
- Queue cap with oldest-first eviction plus a telemetry_dropped counter, so silent loss is itself observable.

Server

- OpenTelemetry: one trace per auction, span per DSP call. In M1 and M2 extend the trace to cover notification firing.
- Metrics for alerting; raw events to a columnar warehouse for funnel joins.
- Ingest dedupes on `event_id`, sized for bursty flushes.

Sampling

- Never sampled: all errors, wins, billing notices, `ad_impression`.
- Sampled: high-volume success events, but record the sampling rate on the event so counts correct upward at query time.

Privacy

- **In M1 Bidon owns consent collection**, and carries the compliance responsibility that goes with it.

- No PII in `error_message`; truncate and scrub.
- GAID gated on `lmt` and consent; send coarse or null rather than dropping the event.
- COPPA-flagged apps: reduced field set.
- Advertising ID: The identifier is populated only where consent is present. Everywhere else the fields carries all-zero UUID. Conditions are evaluated in order and the first match wins. The field is always present and never null, so no downstream consumer has to handle missing key, but a zeroed value is not an identifier and must never be treated as one.

Condition (first match wins)	advertising_id_carries	advertising_id_state
COPPA flagged app or child directed treatment	All-zero UUID	Zeroed_coppa
Consent for personal data is absent , not yet collected or withdrawn	All-zero UUID	Zeroed_no_consent
Limit_ad_tracking or DNT is on, or the user has deleted the advertising ID	All-zero UUID	Zeroed_lat
Consent Present and none of the above	The real GAID	Real

- Retention per event class defined before launch, not after legal review